

Dynamic Polyphony Management, Load Shedding, and Real-Time CPU Budgeting in Multi-Instance Audio Dataflow Graphs

Advanced Academic and Industry Literature Review

Operating interactive digital signal processing (DSP) dataflow graphs under hard real-time constraints requires strict adherence to execution deadlines. In game engine architectures and spatial audio subsystems, the total time allocated to render all active sound sources must remain securely below the hardware audio buffer duration.¹ For example, a 64-sample buffer at 48 kHz permits a maximum processing window of 1.33 ms per callback. Exceeding this budget triggers hardware buffer underflow, resulting in audible dropouts and voice starvation errors.¹ Historically, engines relied on naive priority-based voice culling.⁴ Modern architectures, however, favor predictive, multi-tiered load-shedding and adaptive level-of-detail (LOD) models that gracefully degrade execution complexity before a deadline overrun can manifest.⁶

Anytime Algorithms and Imprecise Computation in Real-Time DSP

The theoretical framework for dynamic degradation in real-time systems is rooted in the paradigm of anytime algorithms and the imprecise computation model.⁸ While traditional run-to-completion algorithms offer no usable output if interrupted prior to termination, anytime algorithms exhibit monotonic increasing utility; the accuracy or quality of the computed result increases as a function of allocated execution time.⁹ Sieve functions, milestone methods, and multiple-version algorithms represent the primary methods for integrating these computational trade-offs into real-time systems.¹⁰ For example, the Newton-Raphson iterative method serves as a classic mathematical archetype of anytime processing.⁸

Under the imprecise computation framework, real-time tasks are split into a mandatory component M_i and an optional component O_i .⁷ The mandatory component must be executed to produce a result of minimum acceptable quality and is scheduled with hard worst-case execution time (WCET) guarantees.⁷ The optional component is executed only if the system possesses sufficient CPU headroom, refining the initial result to maximize output quality.¹¹ This approach has been adapted to signal processing through frameworks such as anytime Fourier analysis, which balances block-oriented delay against spectral precision¹³, and the Adaptive Real-Time Imprecise Computation (ARTIC) pipeline, which tracks stage-by-stage execution times and deadlines to trigger automated precision reductions in audio streams.¹⁴ Fouad and Hahn implemented this paradigm within the Virtual Audio Server (VAS) using a hierarchical representation known as a "Timbre Tree".⁷ A Timbre Tree is a directed structure of

parameterized signal-generating and modifying nodes traversed in a pre-order fashion to synthesize complex waveforms.⁷ The VAS scheduler manages overload conditions by applying the Least Utilization (LU) algorithm to allocate execution time to optional sub-tasks.⁷ The LU algorithm orders optional tasks based on their weighted utilization factor:

$$U_w = \frac{P_i}{U_i}$$

where P_i is the task priority and U_i represents the percentage of available processor time required for full execution.⁷ The scheduler greedily allocates remaining execution time to the

optional tasks in descending order of U_w .⁷

If the available CPU budget is exhausted before a Timbre Tree is fully evaluated, the missing sample blocks are approximated using an Interpolating Buffer (IBuffer), which consists of a stacked configuration of mini-buffers.⁷ Linear interpolation is performed between the closest completed sample blocks to fill the unexecuted gaps.⁷

Despite its mathematical elegance, the LU algorithm exposes significant limitations when applied to continuous real-time audio synthesis:

- **Priority Crossover Artifacts:** Abrupt, distracting fluctuations in sound quality occur when two sounds with similar priorities swap execution states.⁷
- **Lack of Temporal Fairness:** The greedy allocation of optional time means a small subset of high-priority sounds consumes the entire remaining budget, leaving other active voices restricted to their bare minimum quality.⁷
- **Quality Disparities:** Large, audible differences in quality emerge between sounds that have nearly identical priority levels, destroying the cohesion of the soundscape.⁷

Dataflow Graph Scheduling Models and Reconfiguration Complexity

Modern audio engines execute complex, multi-instance dataflow graphs where node topologies and execution paths vary dynamically.¹⁵ Scheduling these systems requires mapping Directed Acyclic Graphs (DAGs) onto shared processor cores.¹⁷ Standard models like Synchronous Dataflow (SDF) and Cyclo-Static Dataflow (CSDF) enforce fixed token production and consumption rates, enabling compile-time scheduling and latency analysis through balance equations¹⁷:

$$c(v) \cdot \alpha(u, v) = c(v') \cdot \beta(u, v')$$

where $c(v)$ represents the execution rate of actor v , while α and β represent token

production and consumption rates.¹⁷ These graphs can be transformed into Affine DataFlow Graphs (ADFGs) to automatically synthesize fixed-priority periodic schedules that minimize buffering and latency.¹⁹

Interactive audio environments exhibit highly dynamic topologies that cannot be modeled solely with static rates. Instead, they require Mode-Controlled Dataflow (MCDF) or the Threaded Abstract Machine (TAM) execution model.²⁰ MCDF captures data-value-dependent behavior while maintaining rigorous temporal analysis and tight resource budgeting.²⁰ TAM provides a framework for task suspension, priorities, and static scheduling across threads.²¹

However, runtime scheduling and synchronization overhead can consume up to 16.3% of the cycle budget on highly constrained embedded processors, although this drops below 1% on high-performance architectures.²⁰ Consequently, the runtime scheduler must execute load-shedding policies with minimal overhead. If a deadline overrun is predicted, the scheduler dynamically reconfigures the graph topology using two primary techniques:

1. **Dynamic Downsampling:** The scheduler inserts a downsampling node at the input of a high-cost sub-graph (such as a complex physical modeling synth or a physical acoustic waveguide) and a matching upsampling node at its output.¹⁶ Halving the execution rate of the sub-graph reduces its CPU consumption by approximately 50%, at the cost of a lower Nyquist limit and potential aliasing.¹⁶
2. **Structural Bypassing:** Computational sub-graphs representing non-essential audio levels-of-detail—such as complex high-frequency air absorption filters or late-reverberation reflection models—are completely bypassed.¹⁶ The signal is routed through simpler, zero-latency polynomial approximations or basic gain stages, ensuring that the critical structural topology of the audio graph is maintained without halting execution.¹⁶

Perceptual Culling and Auditory Saliency Heuristics

To align computational degradation with human perception, modern real-time schedulers incorporate psychoacoustic saliency metrics. Nicolas Tsingos developed advanced perceptual culling and dynamic auditory masking algorithms designed to render complex virtual environments with hundreds of active sound sources.²² The masking thresholds of input signals

are precomputed over three critical frequency bands: low (0–500 Hz), medium (500–2000 Hz), and high (2000 + Hz).²²

At any given execution frame T , the engine evaluates the perceptual loudness L_k^T of each sound source S_k , accounting for distance attenuation, propagation delay $\delta = r/c$, and frequency-dependent attenuation $A_k^T(f)$ caused by obstacles, occlusion, and directivity.²² The perceptual loudness is modeled as:

$$L_k^T = \sum_f \alpha(f) P_k^{T-\delta}(f)$$

where the power spectral distribution is:

$$P_k^{T-\delta}(f) = PSD_k^{T-\delta}(f) \times \frac{A_k^T(f)}{r^2}$$

The weighting coefficients $\alpha(f)$ are empirically derived from equal-loudness contours (typically set to $\alpha(\text{low}) = 0.05$, $\alpha(\text{medium}) = 0.80$, and $\alpha(\text{high}) = 0.15$).²² To prevent high-frequency control-rate modulation, the loudness function is smoothed using a moving average window over 10 frames.²²

Sources are sorted in descending order of a combined loudness and masking metric:

$$H_k = L_k^T \cdot \max(0, 60 - M_k^{T-\delta})$$

where $M_k^{T-\delta}$ is the masking threshold of the source.²² The culling scheduler progressively adds sorted sources to the global mix, updating the dynamic masking threshold T and the accumulated power P_{mix} .²² The iteration terminates when the remaining unmixed power falls below the absolute threshold of hearing (ATH) or is fully masked by P_{mix} :

Po = PTOT

while Po > Pmix - T and Po < ATH do

add source Sk to active mix

Po -= Pk

Pmix += Pk

T += Mk

end

For the sources that survive culling, spatial rendering costs are optimized by grouping them into a constrained number of hardware spatialization channels using a dynamic clustering metric.²²

The spatial error $d(C, S_k)$ between a cluster representative C and a source S_k is defined as:

$$d(C, S_k) = L^{\beta} T_k^{\gamma}$$

where β and γ weight distance and angular errors respectively (empirically set to $\beta = 2, \gamma = 1$).²² The global cluster representative C is computed as a centroid in polar coordinates.²² This avoids the spatial distortion associated with Cartesian centroids, which can pull the representative too close to the listener, artificially inflating the source's loudness.²² Crossmodal visual-auditory scheduling can also be leveraged; visual frustum culling, occlusion tests, and visual-auditory delay tolerance allow the system to deprioritize or downsample audio events that are visually occluded or off-screen.⁶

Volumetric Rendering and Scripted Audio Levels-of-Detail

To bridge the gap between geometric scene representations and DSP execution, Apple patented a 3D audio rendering pipeline utilizing volumetric sound sources and scripted audio levels-of-detail (authored by David Thall, Christopher A. Wolfe, and James E. McCartney).²⁵ Rather than approximating complex physical geometries—such as rivers, highways, or large architectural elements—using arrays of individual point sources (which exponentially scales the number of ray-versus-mesh intersection tests and distance attenuation calculations), this approach models sound sources as volumetric geometric shapes (such as boxes or cylinders) and calculates volumetric acoustic responses.²⁵

The level-of-detail of these volumetric sources is dynamically modified over time using a specialized scripting language.²⁵ This language describes procedurally how the rendering complexity scales as a function of the distance and occlusion between the listener and the volume²⁵:

- **High Proximity (High LOD):** The volumetric source is evaluated with maximum geometric precision. Ray tracing is performed against individual vertices, and multi-point emission arrays are spawned to render spatial width.²⁵
- **Far Proximity (Low LOD):** The source is simplified to a single point source, and complex environmental acoustic path calculations are bypassed.²⁵ Instead of executing full geometric diffraction and absorption paths, the engine falls back to simple distance-based attenuation and basic spectral shaping.²⁵

Real-World Hardware and Power Constraints

While scheduling theory assumes a stable execution platform, real-world deployments must contend with dynamic physical hardware perturbations. In region-specific operational environments characterized by unstable power grids and load shedding, such as Johannesburg, the switching of uninterruptible power supplies (UPS) to battery power can induce voltage fluctuations.³ These fluctuations can trigger CPU thermal throttling, reducing the processor's clock frequency and instantly invalidating static worst-case execution time (WCET) assumptions.³ Under these conditions, an audio engine that was previously stable at a given buffer size may suddenly experience severe underflows and starvations.¹

Additionally, in commercial engines like Wwise (extensively profiled in production titles such as *Scars Above*), registering and unregistering a large number of audio emitters simultaneously is a primary source of severe CPU spikes on eighth-generation and ninth-generation console platforms.¹ To mitigate these real-time scheduling spikes, best practices dictate:

- **Bus-Level Processing:** Heavy plug-ins and DSP effects should be placed on mixed sub-buses rather than individual voice assets.¹ This guarantees that only a single instance of the plug-in is processed per bus channel, regardless of the number of active voices feeding into it.¹
- **Memory-Speed Optimization:** Short, highly repetitive audio files (such as footstep impacts or weapon transients) should be loaded as uncompressed PCM data rather than compressed formats to eliminate decoding CPU overhead on the audio thread, shifting the bottleneck from the CPU to system memory.¹

Algorithmic Policy Taxonomy Matrix

To systematically select a load-shedding and voice-management policy, the compiler and runtime schedulers must balance evaluation complexity, telemetry overhead, and the risk of generating audible artifacts. The table below analyzes these trade-offs across the principal algorithms discussed in the literature.

Strategy Name	Sorting / Evaluation Complexity	Telemetry Accumulation Overhead	Degradation Granularity	Audio Artifact Risk	Underlying Mechanism / Execution Constraint
Naive Volume-Based Culling	$O(N \log N)$	Negligible ($O(1)$ amplitude lookup)	Binary (Kill / Keep)	High (Causes hard signal truncation clicks)	Sources sorted purely by distance-attenuated gain: $G = \frac{1}{d^5}$
Least Utilization (LU) Imprecise Computation	$O(N \log N)$	Moderate (Requires active tracking of mandatory/optional execution times) ⁷	Per-Node / Per-Resolution (Iterative Timbre Tree traversal) ⁷	Medium (High risk of priority crossover and quality disparities) ⁷	Compares weighted utilization factors: $U_w = \frac{\sum_{i=1}^N u_i w_i}{\sum_{i=1}^N w_i}$ Missing samples

					interpolated via stacked sub-buffers. ⁷
Tsingos Auditory Masking	$O(N \log N)$ sorting + $O(N)$ culling pass	Low (Precomputed frequency band power, minimal runtime scaling) ²²	Per-Voice / Per-Instance (Perceptually justified culling) ²²	Very Low (Culled voices are statistically guaranteed to be inaudible) ²²	Progressively evaluates source power against running masking threshold: $P_0 > P_{mix}$ ²²
Polar Coordinate Perceptual Clustering	$O(K \cdot N)$ (where K is target channel count, N is active sources)	Moderate (Requires real-time coordinate transformations to polar space) ²²	Per-Voice / Spatial Grouping	Low (Maintains directionality and average distance, minimizes spatial collapse) ²²	Minimizes spatial error: $d(C, S_k) =$ ²²
Apple Volumetric Scripted LOD	$O(N \log N)$ (Distance sorting)	Low (Calculated on control thread based on geometric proximity) ²⁵	Multi-Tiered (Geometric approximation and path bypassing) ²⁵	Low (Procedural transitions managed by scripts) ²⁵	Simplifies volumetric sources to point sources and bypasses complex propagation calculations as distance increases. ²⁵
Resampling-Based MCDF	$O(N)$ (Dataflow traversal)	Moderate (Tracks local	Per-Node / Subgraph (Halves	Low-to-Medium (Risks aliasing if	Dynamically inserts resampling

degradation	overhead)	execution metrics of individual DAG subgraphs) ¹⁶	sample-rate of targeted branches) ¹⁶	downsampling filters are too steep/shallow) ¹⁶	nodes to lower the internal processing rate of the subgraph. ¹⁶
Kurzweil Layer-Priority Spoofing	$O(1)$	None	Per-Voice / Per-Layer (Gain padding and envelope tracking) ³⁰	Low (Relies on standard voice stealing engine)	Pads algorithmic gain internally by +18 dB while applying an inverse digital pad to trick volume-based stealers. ³⁰
Unison Share Voice Allocation	$O(1)$	None	Per-Voice (Dynamic polyphony distribution) ³¹	Low (Standard polyphonic voice redistribution)	Dynamically redistributes active voice allocations based on the number of concurrent keys depressed. ³¹
Dual-ADSR / Quick-Release Stealing	$O(1)$	Negligible	Per-Voice (Sample-accurate volume attenuation over 6ms window) ²⁹	Near Zero (Effectively eliminates click transients) ²⁹	Overrides standard envelope release phase with a rapid exponential ramp-down

					curve upon voice theft. ²⁹
--	--	--	--	--	---

Transient and Audio Artifact Mitigation

When a real-time voice is instantly terminated or a dataflow graph is reconfigured under strict deadline constraints, the sudden truncation of the continuous waveform produces an abrupt signal discontinuity.²⁸ This step discontinuity acts as an impulse, spreading energy across the spectrum and manifesting as an audible click or pop.²⁸ To prevent these artifacts, real-time engines must manage voice recycling, culling, and state transitions using deterministic, sample-accurate ramp-down techniques.²⁹



The principal low-level DSP mitigation strategies include:

Dual ADSR Envelope Generators

Instead of relying on a single envelope generator per voice, robust synthesis engines implement a dual-envelope topology.²⁹ Each voice instance maintains a primary adsr envelope (which responds to standard MIDI events and gameplay triggers) and an independent, secondary quickRelease envelope generator.²⁹

- **Properties:** The quickRelease envelope is configured with an instantaneous attack/decay time ($0.0s$), a sustain level of 1.0 , and a highly rapid, exponential release time (typically $3\text{ to }6\text{ ms}$, or approximately $144\text{ to }288$ samples at 48 kHz).²⁹
- **Execution Logic:** When a voice is flagged for culling or stealing (i.e., stopNote is invoked with allowTailOff = false), the voice does not immediately terminate.²⁹ It marks its internal state as isFading = true and triggers the quickRelease envelope's release phase.²⁹
- **Rendering:** Within the DSP callback (renderNextBlock), if the isFading flag is active, the engine evaluates the rapid release envelope and multiplies the voice's local buffer by this fading gain before mixing it into the global output bus.²⁹ Once the envelope reaches zero,

the voice is safely cleared, reset, and returned to the inactive voice pool.²⁹

Ramped Buffer Mixing and Sample-Accurate Gain Interpolation

For high-performance dataflow systems, sample-accurate volume ramping is implemented directly at the buffer mixing stage to avoid per-sample envelope evaluation overhead.²⁹ This can be achieved using native buffer ramping APIs:

```
C++
InputBuffer::addFromWithRampChannel( startSample, voiceBuffer, numSamples, 1.0f, 0.0f )
```

This method multiplies the source buffer by a linearly interpolating multiplier that ramps from a start value of $1.0f$ to an end value of $0.0f$ over the course of N samples, blending the result into the output destination.²⁹ If a voice must be stolen midway through a block, the engine processes only the sub-buffer up to the transition frame, applies the ramp-down, and schedules the newly allocated voice to begin processing from that exact sample-accurate offset, preventing any temporal gaps or overlaps in the signal stream.²⁹

Pre-Allocated Allocation-Free Channel Pools

To guarantee hard real-time execution, the voice manager must avoid dynamic memory allocation or garbage collection overhead during gameplay.³² This is achieved by utilizing pre-allocated channel pools (such as the architecture demonstrated by lite-audio-pool).³²

- **Pre-Wiring:** At engine initialization, all audio nodes (such as gain controllers, spatializers, and buffers) are pre-allocated and statically wired.³²
- **State Array Tracking:** The active status and remaining durations of voices are tracked using pre-allocated typed arrays (e.g., Float32Array or static C-style arrays) representing expiration times.³²
- **$O(1)$ Channel Reuse:** When a sound is triggered, the manager scans the pre-allocated state array in $O(1)$ time to find an idle channel.³² If all channels are occupied, the oldest active channel is stolen by initiating a rapid anti-pop gain ramp-down before assigning the new source to the channel.³² This guarantees zero dynamic allocations or memory deallocations during the voice lifetime, eliminating garbage collection spikes or heap fragmentation.³²

Wait-for-Render Synchronization and Legato Transitions

To manage the asynchronous boundary between the gameplay/control thread and the audio rendering thread, engines utilize wait-for-render synchronization.²⁹ When a note is stolen, the control thread sets a thread-safe "stolen" flag on the target voice.²⁹ The synthesis engine is permitted to finish writing its current block—applying a rapid fade-out ramp within the callback—while the initialization of the new generator momentarily yields until the active block is fully rendered, ensuring a seamless, click-free transition.²⁹

Alternatively, the system can bypass voice termination entirely by executing a legato transition.²⁹ Instead of tearing down the entire voice and resetting its filters and oscillators, the voice's pitch, phase, and wavetable pointers are smoothly interpolated to the new target values.²⁹ This preserves the continuous internal state variables of the filters (such as feedback delay networks or physical waveguides), completely avoiding the transient response spikes associated with resetting filter coefficients.²⁹

Telemetry Feedback and Control Architecture

To maintain strict real-time determinism, the telemetry capture phase (audio thread) must be completely decoupled from the policy decision-making phase (control thread).¹⁵ This decoupling is achieved using a unidirectional loop facilitated by lock-free, allocation-free Single-Producer Single-Consumer (SPSC) ring buffers.³³

To measure the overall execution cost of the DSP graph, the system can utilize high-performance, low-overhead driver-level telemetry injection, such as the cross-platform Invisor framework.³⁵ Invisor embeds structured telemetry packets directly into the audio buffer stream, monitoring metrics such as tempo confidence, key detection, harmonic density, transient onset probability, and integrated ^{400 ms} LUFS-S history.³⁵ By utilizing optimized, fixed-point FFT kernels targeted for AVX2 and ARM64 architectures, this telemetry injection layer restricts its CPU overhead to $\leq 1.2\%$, ensuring high-precision tracking with negligible impact on the real-time budget.³⁵

To filter out short-term scheduling noise and transient spikes, the control thread applies a low-pass Exponential Moving Average (EMA) filter to the execution times gathered from the telemetry queue⁷:

$$\tau_{\text{pred}}[n] = \alpha \cdot \tau_{\text{meas}}[n] + (1 - \alpha) \cdot \tau_{\text{pred}}[n - 1]$$

where $\alpha \in (0, 1]$ is the smoothing factor (typically set to $\alpha = 0.2$ to balance responsiveness against noise rejection). If the predicted execution time exceeds the target budget ($\sum \tau_{\text{pred}} > \gamma \cdot T_{\text{budget}}$, where $\gamma = 0.85$ is a safety headroom factor), the control thread triggers its load-shedding algorithms.¹ It formulates target level-of-detail and

downsampling configurations, packaging them as atomic commands to be dispatched back to the audio thread via the SPSC Control Queue.³³

```
C++
#include <atomic>
#include <array>
#include <cstdint>

// Fixed system configurations
constexpr size_t MAX_VOICES = 128
constexpr size_t RING_BUFFER_CAPACITY = 256

// Structures passed between threads
struct TelemetryPacket {
    uint32_t voiceId;
    uint64_t nanosecondsElapsed;
    float peakLoudness;
    uint32_t activeNodes;
};

struct ThrottlingCommand {
    uint32_t voiceId;
    float targetVolume;
    bool enableDownsampling;
    bool bypassHighCostNodes;
};

// Lock-free, allocation-free SPSC Ring Buffer
template <typename T, size_t Capacity>
class LockFreeSPSC {
    static_assert(Capacity & (Capacity - 1) == 0 "Capacity must be a power of 2")
public:
    LockFreeSPSC() : writeIndex(0), readIndex(0) {}

    bool push(const T& item) noexcept {
        const size_t currentWrite = writeIndex.load(std::memory_order_relaxed);
        const size_t currentRead = readIndex.load(std::memory_order_acquire);
```

```

        if ((currentWrite - currentRead) >= Capacity)
            return false // Buffer full (discarding telemetry is preferable to blocking)
    }

    buffer = item;
    writeIndex.store(currentWrite + 1, std::memory_order_release);
    return true;
}

bool pop(T& item) noexcept
{
    const size_t currentRead = readIndex.load(std::memory_order_relaxed);
    const size_t currentWrite = writeIndex.load(std::memory_order_acquire);

    if (currentRead == currentWrite)
        return false // Buffer empty
    }

    item = buffer;
    readIndex.store(currentRead + 1, std::memory_order_release);
    return true;
}

private:
    std::array<T, Capacity> buffer;
    alignas(64) std::atomic<size_t> writeIndex;
    alignas(64) std::atomic<size_t> readIndex;
}

// Real-Time Audio Callback Context
class RealTimeAudioEngine {
public:
    RealTimeAudioEngine() = default;

    // Process loop executed directly by the system hardware audio thread
    void processAudioCallback(float* outputBuffer, size_t sampleCount) noexcept
    {
        // Step 1: Drain control queue and apply throttling parameters
        ThrottlingCommand command;
        while (controlQueue.pop(command))
            applyThrottlingToVoice(command);
    }

    // Step 2: Render active dataflow graph instances and measure performance
    for (size_t i = 0; i < MAX_VOICES; ++i)

```

```

        if (!voices[i].isActive) continue;

        const uint64_t timeStart = queryPlatformTimer();

        // Execute the voice DSP graph
        voices[i].dspGraph.processBlock(voices[i].localBuffer, sampleCount);

        const uint64_t timeEnd = queryPlatformTimer();
        const uint64_t timeElapsed = timeEnd - timeStart;

        // Step 3: Mix voice output into global buffer using sample-accurate ramps
        mixVoiceToOutput(outputBuffer, voices[i], sampleCount);

        // Step 4: Dispatch execution telemetry (lossy, non-blocking)
        TelemetryPacket packet;
        packet.voiceId = static_cast<uint32_t>(i);
        packet.nanosecondsElapsed = timeElapsed;
        packet.peakLoudness = voices[i].dspGraph.getPeakLoudness();
        packet.activeNodes = voices[i].dspGraph.getActiveNodeCount();
        packet.y = y;
        telemetryQueue.push(packet);
    }

    // High-resolution clock query using platform-specific APIs
    static inline uint64_t queryPlatformTimer() noexcept
    #if defined(__APPLE__)
    {
        return mach_absolute_time();
    }
    #elif defined(_WIN32)
    {
        LARGE_INTEGER ticks;
        QueryPerformanceCounter(&ticks);
        return static_cast<uint64_t>(ticks.QuadPart);
    }
    #else
    {
        struct timespec ts;
        clock_gettime(CLOCK_MONOTONIC, &ts);
        return static_cast<uint64_t>((ts.tv_sec * 1000000000ULL +
static_cast<uint64_t>(ts.tv_nsec)
    }
    #endif

private:
    struct VoiceInstance {
        bool isActive = false;

```

```

float localBuffer;

struct DSPGraph {
    void processBlock(float* buf, size_t count) noexcept
        // Iteratively evaluates parameterized node trees (e.g., Timbre Trees)

    float getPeakLoudness() const noexcept { return 0.707f; }
    uint32_t getActiveNodeCount() const noexcept { return 12; }
} dspGraph;

void applyThrottlingToVoice(const ThrottlingCommand& cmd) noexcept
{
    if (cmd.voiceId < MAX_VOICES)
        // Apply parameters safely in a lock-free manner
        // Modifies internal sample-rate downsampling and bypasses high-cost DSP nodes

    void mixVoiceToOutput(float* out, const VoiceInstance& voice, size_t count) noexcept
        // Blends active voices using sample-accurate ramping or zero-crossing crossfades

    VoiceInstance voices;

public:
    LockFreeSPSC<TelemetryPacket, RING_BUFFER_CAPACITY> telemetryQueue;
    LockFreeSPSC<ThrottlingCommand, RING_BUFFER_CAPACITY> controlQueue;

```

Architectural Synthesis and Nuanced Engineering Recommendations

To deploy a high-performance load-shedding and adaptive scheduling architecture in production environments, several platform-specific and memory-level guidelines should be followed.

Dynamic Core Partitioning and Thread Affinity

The system must guarantee that the real-time audio thread runs at the highest possible scheduler priority (e.g., real-time priority 97 via `pthread_setschedparam` using `SCHED_FIFO` on POSIX systems, or thread priority `THREAD_PRIORITY_TIME_CRITICAL` on Windows). The asynchronous control thread should be configured with standard, non-real-time priority (`SCHED_OTHER` / `THREAD_PRIORITY_NORMAL`) to prevent scheduling conflicts.

Furthermore, on multi-core systems, the control thread should be explicitly pinned to a separate

physical core using CPU affinity masks:

```
C++
cpu_set_t cpuset;
CPU_ZERO(&cpuset);
CPU_SET(control_core_id, &cpuset);
pthread_setaffinity_np(control_thread_native_handler, sizeof cpu_set_t, &cpuset);
```

By isolating the control thread's workloads (such as spatial sorting, polar coordinates transformation, and clustering algorithms) to a separate core, the system prevents cache pollution and scheduling jitter on the core dedicated to processing the real-time audio callback.

Cache-Line Alignment and False Sharing Mitigation

To prevent memory bus contention and CPU stall cycles within the SPSC queues, the atomic read and write indices must be explicitly aligned to the CPU's cache-line boundaries (typically 64 bytes on modern architectures).³⁴ Failing to align these variables leads to false sharing, where the CPU core executing the audio callback and the core executing the control thread repeatedly invalidate each other's L1 cache lines.³⁴ This can be resolved by using explicit alignment attributes:

```
C++
alignas(64) std::atomic<size_t> writeIndex;
alignas(64) std::atomic<size_t> readIndex;
```

This alignment guarantees that the atomic write and read pointers reside on completely separate cache lines, allowing both cores to read and write concurrently without causing memory pipeline stalls.³⁴

Dynamic Profiling and Self-Calibrating Predictors

Because CPU performance varies dynamically due to thermal throttling, background OS processes, and hardware power states, hard-coded execution time estimates are unreliable.³ Real-time systems should utilize self-calibrating performance profiles.⁸ The control thread

should continuously monitor the variance between its predicted execution times (τ_{pred}) and the actual measured durations (τ_{meas}) popped from the telemetry queue. If the prediction error exceeds a predefined threshold:

$$|\tau_{\text{meas}} - \tau_{\text{pred}}| > \epsilon_{\text{threshold}}$$

the smoothing coefficient α should be dynamically increased to adapt to the changing performance characteristics of the host system. This self-calibrating loop prevents the engine from over-throttling during temporary performance dips and ensures that the system reacts quickly to persistent resource limitations.

Works cited

1. Audio Optimization Practices in Scars Above - Audiokinetic, accessed May 22, 2026, <https://www.audiokinetic.com/en/community/blog/audio-optimization-practices-in-scars-above/>
2. Audio stutters and cuts out - Unreal Engine - FMOD Forums, accessed May 22, 2026, <https://qa.fmod.com/t/audio-stutters-and-cuts-out/16503>
3. Logic Pro CPU Overload in Johannesburg: Why Your Mac Cr - ZA Support, accessed May 22, 2026, <https://zasupport.com/blog/logic-pro-cpu-overload-johannesburg>
4. US5376752A - Open architecture music synthesizer with dynamic voice allocation - Google Patents, accessed May 22, 2026, <https://patents.google.com/patent/US5376752A/en>
5. Game Audio Coding vs. Aesthetics, accessed May 22, 2026, <https://videogameaudio.com/GDC-Mar2003/GDC2003-CodingVsAesthetics-Slides-LPaul.pdf>
6. Sound Level of Detail in Interactive Audiographic 3D Scenes - ResearchGate, accessed May 22, 2026, https://www.researchgate.net/publication/256436694_Sound_Level_of_Detail_in_Interactive_Audiographic_3D_Scenes
7. Perceptually Based Scheduling Algorithms for Real-time Synthesis ..., accessed May 22, 2026, https://icg.gwu.edu/sites/g/files/zaxdzs6126/files/downloads/FouadHahn1997_0.pdf
8. Anytime algorithm - Wikipedia, accessed May 22, 2026, https://en.wikipedia.org/wiki/Anytime_algorithm
9. Exploiting Anytime Algorithms for Collaborative Service Execution in Edge Computing, accessed May 22, 2026, <https://www.mdpi.com/2073-431X/13/6/130>
10. Anytime Algorithms for Automotive Architectures UTC Final Report - ROSA P,

- accessed May 22, 2026,
https://rosap.ntl.bts.gov/view/dot/36246/dot_36246_DS1.pdf
11. Anytime Algorithms for GPU Architectures, accessed May 22, 2026,
<https://www.infor.uva.es/~yuri.torres/docs/06121425.pdf>
 12. Design of Embedded Controllers Based on Anytime Computing - Centro di Ricerca Enrico Piaggio, accessed May 22, 2026,
https://www.centropiaggio.unipi.it/sites/default/files/2010_QFGPB_TII.pdf
 13. View of Anytime Fourier Analysis - Periodica Polytechnica - BME, accessed May 22, 2026, <https://pp.bme.hu/ee/article/view/907/525>
 14. Work-in-Progress: ARTIC: An Adaptive Real-Time Imprecise Computation Pipeline for Audio Analysis - IEEE Xplore, accessed May 22, 2026,
<https://ieeexplore.ieee.org/document/9052142/>
 15. Real-time scheduling for interactive multimedia systems, accessed May 22, 2026,
<https://perso.eleves.ens-rennes.fr/~pdonatbo/documents/info/masterReport.pdf>
 16. Work in Progress: Adaptive Scheduling with Approximate Computing for Audio Graphs - Christoph Kirsch, accessed May 22, 2026,
<https://ckirsch.github.io/publications/conferences/RTSSWIP17-AdaptiveScheduling.pdf>
 17. Strictly Periodic Scheduling of Acyclic Synchronous Dataflow Graphs using SMT Solvers - [Verimag], accessed May 22, 2026,
<https://www-verimag.imag.fr/TR/TR-2014-5.pdf>
 18. Parallel Scheduling of Multiple SDF Graphs Onto Heterogeneous Processors - IEEE Xplore, accessed May 22, 2026,
<https://ieeexplore.ieee.org/iel7/6287639/9312710/09335930.pdf>
 19. Real-Time Fixed Priority Scheduling Synthesis using Affine DataFlow Graphs: from Theory to Practice | Request PDF - ResearchGate, accessed May 22, 2026,
https://www.researchgate.net/publication/373228760_Real-Time_Fixed_Priority_Scheduling_Synthesis_using_Affine_DataFlow_Graphs_from_Theory_to_Practice
 20. Temporal analysis and scheduling of hard real-time radios running on a multi-processor - Pure, accessed May 22, 2026,
<https://pure.tue.nl/ws/files/3517710/724538.pdf>
 21. Non-preemptive real-time scheduling of dataflow systems - SciSpace, accessed May 22, 2026,
<https://scispace.com/pdf/non-preemptive-real-time-scheduling-of-dataflow-systems-3go6y01qib.pdf>
 22. Perceptual Audio Rendering of Complex Virtual Environments - Inria, accessed May 22, 2026, <https://www-sop.inria.fr/reves/Nicolas.Tsingos/publis/RR-4734.pdf>
 23. Efficient and Practical Audio-Visual Rendering for Games using Crossmodal Perception - Inria, accessed May 22, 2026,
<https://www-sop.inria.fr/reves/Basilic/2009/GBWAD09/crossmodal13D09.pdf>
 24. Bimodal perception of audio-visual material properties for virtual environments - Inria, accessed May 22, 2026,
<http://www-sop.inria.fr/reves/Basilic/2010/BSVD10/soundingBRDF.pdf>
 25. Apple Patent | 3d Audio Rendering Using Volumetric Audio Rendering And Scripted Audio Level-Of-Detail, accessed May 22, 2026,

- <https://patent.nweon.com/13042>
26. Speech Signal Processing Patents and Patent Applications (Class 704/200), accessed May 22, 2026, <https://patents.justia.com/patents-by-us-classification/704/200>
 27. Reverberators Patents and Patent Applications (Class 381/63) - Justia Patents Search, accessed May 22, 2026, <https://patents.justia.com/patents-by-us-classification/381/63>
 28. Making a Music System – Part 1 - Designing Sound, accessed May 22, 2026, <https://designingsound.org/2016/08/31/making-a-music-system-part-1/>
 29. Voice steal pops - Page 2 - General JUCE discussion - JUCE, accessed May 22, 2026, <https://forum.juce.com/t/voice-steal-pops/30923?page=2>
 30. How can I control voice allocation, so I can choose which programs will... - Sweetwater, accessed May 22, 2026, <https://www.sweetwater.com/sweetcare/articles/control-voice-allocation-so-choose-programs-voices-stolen-run-2/>
 31. Assign Mode - Cherry Audio Documentation, accessed May 22, 2026, <https://docs.cherryaudio.com/cherry-audio/instruments/mercury-4/assign-mode>
 32. GitHub - PeshoVurtoleta/lite-audio-pool: Zero-GC Web Audio sound sprite pool. Voice stealing with 20ms anti-pop fade, pitch shifting, stereo pan., accessed May 22, 2026, <https://github.com/PeshoVurtoleta/lite-audio-pool>
 33. Concurrency — list of Rust libraries/crates // Lib.rs, accessed May 22, 2026, <https://lib.rs/concurrency>
 34. Fikoko/Canon-C: A semantic standard library for C — explicit, composable modules that add meaning without hiding behavior. - GitHub, accessed May 22, 2026, <https://github.com/Fikoko/Canon-C>
 35. Invisor Displays Detailed Information for Your Music and Optimizes Playback Efficiency, accessed May 22, 2026, <https://lifetips.alibaba.com/tech-efficiency/invisor-displays-detailed-information-for-your-music-an>